Decision-OS V11 Note: Forget for Evolution

Reconnectable Forgetting for Agentic AI

Shinichi Nagata

Abstract

Agentic AI systems increasingly operate over long histories: previous conversations, files, tool calls, code changes, execution traces, and human feedback. As autonomy increases, the system is often forced to re-read or re-contextualize more of its past at every step. This creates a structural failure mode: token cost and context load grow faster than useful intelligence.

This note introduces **Reconnectable Forgetting** as a minimal memory principle for agentic AI. The core claim is that self-evolving AI does not require perfect memory. It requires a memory structure that can forget safely while preserving the ability to reconnect later.

Reconnectable Forgetting treats forgetting not as loss, but as controlled compression. Raw history is compressed into reusable structure, anchored by evidence and hashes, and indexed by time so that future intelligence can re-evaluate the original context when needed. The goal is not to remember everything, but to forget in a way that remains auditable, reversible, and re-enterable.

In the Decision-OS lineage, V11 follows V10's Survival-Bounded Planning. V10 asked how an agent survives without breaking its aspiration. V11 asks what the surviving agent must stop carrying in full in order to continue evolving.

---

1. Introduction

As AI agents become more autonomous, memory becomes both a capability and a burden.

A simple assistant can answer from a short prompt. A long-running agent cannot. It accumulates conversations, files, code edits, decisions, execution traces, user preferences, failed attempts, and partial plans. The more

autonomous the agent becomes, the more it must decide what previous state matters.

The naive solution is to remember everything.

However, remembering everything creates its own failure. If the agent must repeatedly re-read full logs, files, and histories, the cost of continuation grows with the history itself. More context does not automatically produce better judgment. At some point, the agent spends increasing resources reconstructing its own past rather than moving forward.

The problem is therefore not simply memory shortage.

The problem is the absence of reusable forgetting.

Long-term agents require a memory structure that does not preserve all information in full, but still preserves enough structure, evidence, and temporal coordinates for future re-evaluation.

This note calls that principle **Reconnectable Forgetting**.

---

2. Question

Why does self-evolving AI require forgetting?

The ordinary framing treats memory as a capacity problem: if the system can store more, retrieve more, and search more, then long-term intelligence improves.

But long-term agency creates a different problem. The system must not only store the past. It must decide what past information must be reloaded, what can be compressed, what can be ignored, and what must remain reconnectable.

This produces the central question of V11:

What kind of forgetting allows an AI agent to evolve without losing the ability to re-evaluate its past?

The answer is not deletion.

The answer is reconnectable compression.

---

3. Core Claim

Self-evolving AI does not need to remember everything.

It needs to forget in a way that can be reconnected later.

More precisely:

Evolution requires forgetting, but forgetting must remain reconnectable.

A system that remembers everything may become too expensive to operate.

A system that forgets without structure may lose accountability, coherence, and recoverability.

A system that compresses without evidence may hallucinate its own history.

Therefore, the required memory form is neither total memory nor ordinary deletion.

It is controlled forgetting with three preserved anchors:

1. reusable structure,
2. evidence of origin,
3. temporal coordinates for re-entry.

This is the minimal claim of Reconnectable Forgetting.

---

4. Key Definition

Reconnectable Forgetting is the controlled compression of past information in which raw history is not fully preserved, but the structure, evidence, and time coordinates required for future re-evaluation remain available.

In short:

Reconnectable Forgetting = forgetting that preserves the path back.

It differs from ordinary forgetting in three ways.

First, it is not deletion. The system may discard raw detail, but it preserves the structural residue needed for later use.

Second, it is not summary alone. A summary without evidence can become a new fiction. Reconnectable Forgetting requires anchors that allow the compressed structure to be checked against its origin.

Third, it is not static archiving. The point is not to store everything in cold memory. The point is to preserve a re-entry path that lets future intelligence reconnect when the structure becomes relevant again.

---

5. Minimal Architecture

Reconnectable Forgetting requires only three operations.

5.1 Compress for Use

The first operation is compression for use.

Raw logs are not the unit of long-term intelligence. They are too large, too noisy, and too dependent on local context.

The system must compress raw history into reusable structures:

- claims,
- decisions,
- constraints,
- failure modes,
- unresolved residues,
- evidence pointers,
- recheck triggers.

The compression must be task-relevant, not merely shorter. A shorter text that removes the decision structure is not useful compression. A useful compression preserves what future action needs.

The output is not “what happened.”

The output is “what must remain usable.”

5.2 Hash for Truth

The second operation is evidence anchoring.

Compression creates risk. A compressed memory can distort the past, smooth over uncertainty, or replace evidence with a persuasive narrative.

Therefore, compressed memory must be anchored.

At minimum, the system should preserve:

- source reference,
- As-of timestamp,
- version or commit identifier,

- artifact hash when available,
- evidence pointer or retrieval path.

The purpose of hashing is not to store all content in the active context. The purpose is to make the compression auditable. If future intelligence needs to verify the compressed claim, it must be able to reconnect to the original evidence or prove that the evidence identity has not changed.

Without this layer, forgetting becomes narrative rewriting.

5.3 TimeTube for Retrieval

The third operation is temporal indexing.

A compressed structure must retain when and under what context it emerged. The same statement can mean different things at different times, under different constraints, tools, model capabilities, and objectives.

Therefore, Reconnectable Forgetting requires a time coordinate.

This does not mean every detail must be reloaded. It means the system preserves enough temporal information to answer:

- When was this structure formed?
- Under what constraints?
- Under what capability ceiling?
- What was unknown at the time?
- What future event should trigger re-evaluation?

This connects V11 to the TimeTube view: memory is not a pile of stored text, but a set of re-enterable coordinates along a trajectory.

---

6. Link to the Decision-OS Lineage

V11 does not replace the earlier layers. It depends on them.

The lineage can be summarized as follows:

V6: PIC / Canon

A structure can be merged without destructive inconsistency.

V7: Self-Evolution Gate

A system can update itself only when direction, boundary, and stable recursion are present.

V8: TimeTube

Evaluation must move from point outputs to trajectories over time.

V9: As-of / Seat / Release

Past decisions must be fixed at their time of execution, responsibility must remain seated, and public release must pass through impact-weighted gates.

V10: Survival-Bounded Planning

When the original goal-length becomes non-survivable, the trajectory must be rescaled while preserving aspiration.

V11: Reconnectable Forgetting

After survival is preserved, the agent must decide what not to carry forward in full, while preserving the ability to reconnect later.

In this lineage, V11 is the memory economy layer.

V10 asks:

What must be rescaled so the agent can survive?

V11 asks:

What must be forgotten so the agent can continue evolving?

---

7. Key Distinction: Loss vs. Reconnectable Loss

The central distinction of V11 is:

Unanchored forgetting is loss.
Anchored forgetting is compression.
Reconnectable forgetting is evolution-ready compression.

If raw history is deleted without structure, the system loses recoverability.

If raw history is summarized without evidence, the system risks hallucinated continuity.

If raw history is compressed into structure, anchored by evidence, and indexed by time, then forgetting becomes a useful operation.

This is why forgetting can be part of evolution.

An evolving system must not carry every previous state at full resolution. It must carry what allows future reconstruction, re-evaluation, and reuse.

In this sense, forgetting is not the opposite of memory. It is a memory operation.

---

8. Implications

8.1 Long-term agency becomes cheaper

If an agent can operate from compressed structures rather than full histories, token cost and retrieval cost decrease.

The system no longer needs to re-read everything. It needs to know what to reload, when to reload it, and why.

8.2 Memory becomes auditable

Reconnectable Forgetting prevents compression from becoming unchecked narrative drift.

By preserving evidence anchors and time coordinates, the system can distinguish between:

- what was actually fixed,
- what was later inferred,
- what remains uncertain,
- and what must be rechecked.

8.3 Forgetting becomes reversible enough

The framework does not require perfect reversibility. It requires sufficient re-entry.

The raw past may not always be fully reconstructed, but the future system must retain enough structure to reconnect to the relevant evidence or admit that reconnection is no longer possible.

8.4 Self-evolution becomes less dependent on total context

A self-evolving system cannot load its entire developmental history at every update. It needs compressed residues that preserve the parts of the past that matter for future evolution.

Thus, Reconnectable Forgetting is not merely a storage optimization. It is a condition for long-horizon self-evolution.

---

9. Falsifier

This note is wrong if the following condition consistently holds:

Long-term AI agents can continue re-reading full histories, files, and action traces without compression, forgetting, or reconnectable memory structures, while maintaining both cost efficiency and decision quality.

In Japanese:

長期エージェントが、履歴を圧縮・忘却・再接続せず、全履歴を再読し続けても、コスト効率と判断品質を維持できるなら、この理論は成立しない。

This falsifier matters because V11 must not become a metaphor about memory.

It is an operational claim. If total recall remains cheaper, more accurate, and more scalable than reconnectable compression, then Reconnectable Forgetting loses its reason to exist.

The current direction appears to be the opposite: agentic systems become more expensive and more fragile as histories grow without structured compression.

---

10. Scope Boundary

This note is intentionally narrow.

It does not define a complete memory architecture.

It does not propose a specific vector database, retrieval engine, compression algorithm, or agent framework.

It does not solve all problems of long-term memory, privacy, identity, or forgetting ethics.

It does not claim that raw data should always be deleted.

It does not claim that summaries are sufficient.

V11 only defines the minimal principle:

A self-evolving agent must be able to forget without losing the path back.

Future work may formalize retrieval policies, compression formats, evidence standards, and evaluation protocols. Those are outside the scope of this note.

11. Conclusion

Agentic AI does not fail only because it cannot reason. It can also fail because it cannot stop re-reading itself.

As histories grow, the central memory problem shifts from storage to reuse. A long-term agent must decide what to keep active, what to compress, what to archive, and what to forget.

Decision-OS V11 introduces Reconnectable Forgetting as a minimal principle for this problem.

The principle is simple:

Do not remember everything.

Do not forget blindly.

Compress what matters, anchor it to evidence, and preserve the path for future re-entry.

Or, in one line:

Evolution requires forgetting, but forgetting must remain reconnectable.

In Japanese:

進化には忘却が必要である。ただし、その忘却は未来から再接続できなければならない。